

CS24B045 | LAB-12 REPORT

Table 1: Comparison between original and interchanged versions

Kernel	Version	Cycles	Instructions	IPC	CPI	Cache Miss Rate
Matmul	Original	43,272,745	109,515,945	2.53	0.40	16.69
Matmul	Interchanged	60,593,017	138,340,538	2.28	0.44	1.74
Transpose	Original	6,532,885	4,247,819	0.65	1.54	7.7
Transpose	Interchanged	3,387,924	3,387,924	0.7	1.43	2.7

Analysis Questions:

- How does loop interchange affect locality?

Matmul:

In a transpose operation ($B[j][i]=A[i][j]$), one matrix is always accessed by row while the other is accessed by column. Loop interchange merely swaps which matrix is accessed contiguously, so the impact on overall locality is limited compared to Matmul.

Transpose:

Here in the original code for one matrix if it is reading row by row then for the other matrix it edits by column by column also in the interchanged one for one matrix it reads column by column and for the other matrix it edits row by row so there isn't much change.

- Which ordering improves cache behavior?

Matmul:

As the smoothness varies to the interchanged one so the cache miss rate improves very high.

Transpose:

Cache behavior remains poor. Because the innermost loop still jumps across memory addresses rather than reading them contiguously, the CPU will continue to experience high cache miss rates.

- How does IPC change after transformation?

Matmul:

The IPC (Instructions Per Cycle) increases after transformation. This happens because the CPU spends less time "stalled" waiting for data from memory and more time executing instructions.

Transpose:

The IPC for Transpose is "consistently low". While a small improvement might be seen depending on how the hardware handles writes vs. reads, the change is generally less dramatic than in Matmul.

- Does instruction count change significantly? Why?

Matmul:

The instruction count does not change significantly. The kernel still performs the same $O(N^3)$ multiplications and additions; only the order of memory access changes.

Transpose:

The instruction count remains nearly identical as the kernel performs N^2 assignments regardless of the loop

order .

- **Which kernel is more sensitive to loop ordering?**

Matmul:

Matmul is highly sensitive to loop ordering. Performance and IPC drop significantly as the matrix size increases if the ordering is inefficient.

Transpose:

Transpose is less sensitive to simple loop interchange. Because you cannot make both matrices contiguous at once, simple reordering does not solve the underlying memory bottleneck.

- **Does loop interchange shift the kernel toward compute-bound or memory-bound behavior?**

Matmul:

Loop interchange shifts the kernel from being memory-bound (limited by data access speed) toward being compute-bound (limited by CPU speed).

Transpose:

Transpose remains memory-bound even after interchange. The low IPC indicates the hardware is primarily waiting for memory sub-systems to move data.